

lab3-3 基于 UDP 服务设计可靠传输协议并编程实现

才子桓 2212700 密码科学与技术

实验要求

实验3-1： 利用数据报套接字在用户空间实现面向连接的可靠数据传输，功能包括：建立连接、差错检测、确认重传等。流量控制采用停等机制，完成给定测试文件的传输。

实验3-2： 在实验 3-1 的基础上，将停等机制改成基于滑动窗口的流量控制机制，采用固定窗口大小，支持累积确认，完成给定测试文件的传输。

实验3-3： 在实验 3-2 的基础上，选择实现一种拥塞控制算法，也可以是改进的算法，完成给定测试文件的传输。

实验3-4： 基于给定的实验测试环境，通过改变网路的延迟时间和丢包率，完成下面3组性能对比实验：

- (1) 停等机制与滑动窗口机制性能对比；
- (2) 滑动窗口机制中不同窗口大小对性能的影响；
- (3) 有拥塞控制和无拥塞控制的性能比较。

要求如下：

1. 实现单向传输。
2. 对于每一个任务要求给出详细的协议设计。
3. 给出实现的拥塞控制算法的原理说明。
4. 完成给定测试文件的传输，显示传输时间和平均吞吐率。
5. 性能测试指标包括吞吐率、文件传输时延等，给出图形结果并进行分析。
6. 完成详细的实验报告（每个任务完成一份）。
7. 编写的程序应该结构清晰，具有较好的可读性。
8. 提交程序源码、可执行文件和实验报告。

实验说明

本次实验在实验 3-1 和 3-2 的基础上，选择实现一种拥塞控制算法，这里我选择 TCP 的 RENO 算法实现拥塞控制。

实验环境

编译IDE： Visual Studio 2022

编译语言： C++ 14标准

lab3 测试文件

路由器 Router

本实验在 Windows 系统下进行。

实验分析及代码实现

这里仅展示对比 3-2 中改动的部分。

RENO 拥塞控制算法

RENO 算法有三个核心状态：

- SLOW_START（慢启动）：连接初始状态，cwnd 从 1 开始指数增长。
- CONGESTION_AVOIDANCE（拥塞避免）：当 cwnd 达到 ssthresh 时进入，cwnd 线性增长。
- FAST_RECOVERY（快速恢复）：发生轻微拥塞时进入，快速调整 cwnd。

1. 慢启动阶段：

- 初始状态下，cwnd 为 1。
- 每收到一个 ACK，cwnd 增加 1。
- 当 cwnd 达到 ssthresh 时，进入拥塞避免阶段。

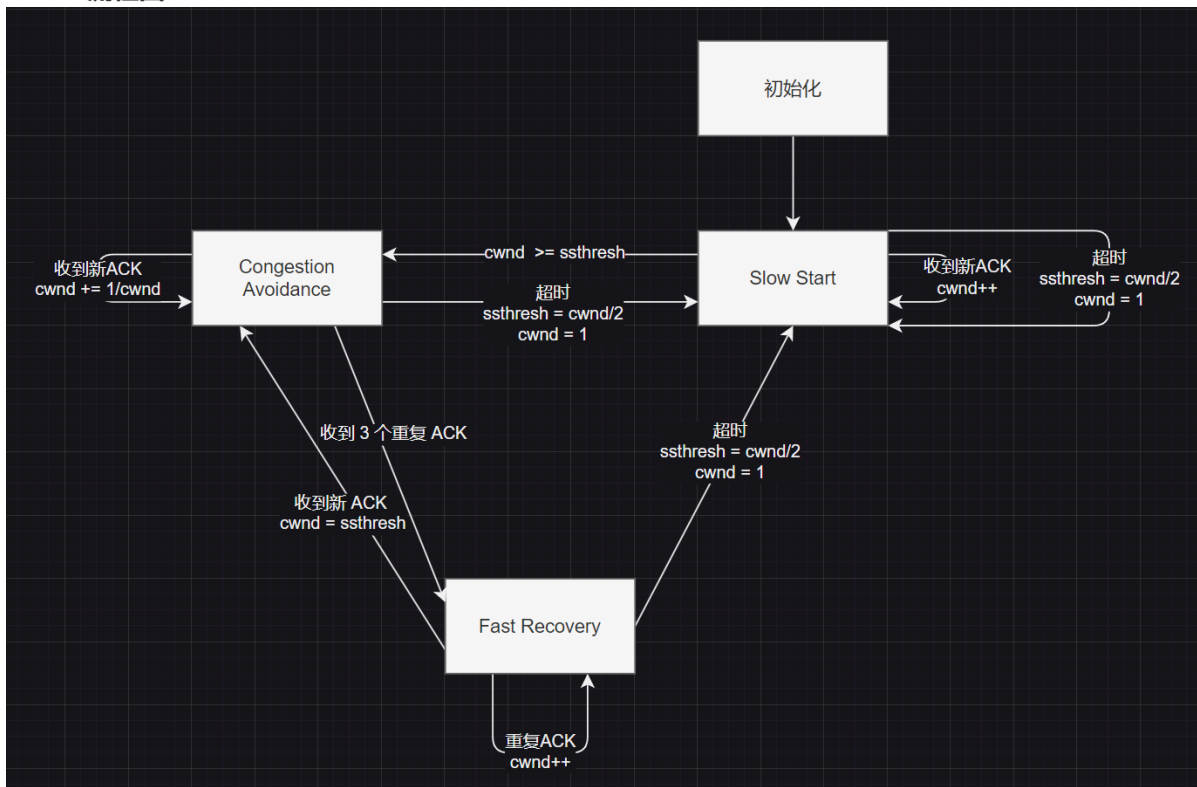
2. 拥塞避免阶段：

- 每收到一个完整的 cwnd 个 ACK，cwnd 增加 1。
- 使用 cwnd_increment 实现平滑的线性增长。
- 增长速度比慢启动慢，避免网络拥塞。

3. 快速恢复阶段：

- 收到 3 个重复 ACK 时进入。
- 立即重传丢失的报文段。
- 将 ssthresh 设为 cwnd 的一半。
- cwnd 设为新的 ssthresh + 3。

RENO 流程图：



代码修改

这里仅展示对比实验 3-2 中改动的部分。

在实验 3-2 的基础上，添加了一个新的枚举类型 CongestionState，用于表示当前的拥塞状态。SLOW_START 慢启动阶段，CONGESTION_AVOIDANCE 拥塞避免阶段，FAST_RECOVERY 快速恢复阶段。

```
enum CongestionState { SLOW_START, CONGESTION_AVOIDANCE, FAST_RECOVERY };
```

定义在拥塞控制中需要的变量。

```
int cwnd = INIT_CWND;           // 拥塞窗口，控制发送方发送速率
int ssthresh = INIT_SSTHRESH;   // 慢启动阈值，控制状态转换点
int duplicate_acks = 0;          // 重复ACK计数，用于检测网络轻微拥塞
float cwnd_increment = 0.0;      // 拥塞避免阶段的cwnd增量累计
```

修改 rcvackthread 函数，加入拥塞控制逻辑。

慢启动阶段，每收到一个ACK，cwnd加1，当cwnd达到阈值ssthresh后，转换状态进入拥塞避免阶段。

```
if (congestion_state == SLOW_START) {
    cwnd++; // 每收到一个ACK，cwnd加1
    if (cwnd >= ssthresh) {
        congestion_state = CONGESTION_AVOIDANCE; // 达到阈值后转换状态
    }
}
```

拥塞避免阶段，每收到一个完整的cwnd个ACK，cwnd增加1，使用cwnd_increment实现平滑的线性增长。

```
else if (congestion_state == CONGESTION_AVOIDANCE) {
    cwnd_increment += 1.0 / cwnd; // 累积增量
    if (cwnd_increment >= 1.0) { // 当累积达到1时增加cwnd
        cwnd++;
        cwnd_increment -= 1.0;
    }
}
```

快速恢复阶段，收到3个重复ACK时进入。立即重传丢失的报文段，将ssthresh设为cwnd的一半，cwnd设为新的ssthresh+3。

```
if (duplicate_acks == 3) {
    if (congestion_state != FAST_RECOVERY) {
        ssthresh = cwnd / 2; // 将慢启动阈值减半
        cwnd = ssthresh + 3; // 设置新的拥塞窗口
        congestion_state = FAST_RECOVERY;
        resend = true; // 触发快速重传
    }
}
else if (congestion_state == FAST_RECOVERY) {
    cwnd++; // 快速恢复期间每收到重复ACK增加cwnd
}
```

超时处理。

当发送窗口内的报文段没有收到ACK时，触发超时处理。将慢启动阈值减半，重置拥塞窗口为1，返回慢启动状态。同时ssthresh减半，降低网络负载。

```
if (clock() - messagestart > MAX_WAIT_TIME) {  
    ssthresh = cwnd / 2;    // 将慢启动阈值减半  
    cwnd = 1;              // 重置拥塞窗口  
    congestion_state = SLOW_START; // 返回慢启动状态  
    // 重传数据  
}
```

实际发送控制中窗口取流量控制窗口和拥塞窗口的最小值，确保不会发送超过网络承载能力的数据。

```
int effective_window = min(window_size, cwnd);  
if (arrive < start + effective_window && arrive < nummessage + 2) {  
    ...// 发送数据  
}
```

实验结果

端口设置：

```
ServerPORT = 112  
ClientPORT = 110  
RouterPORT = 111
```

Router设置：

The image shows a 'Router' configuration window with the following settings:

- 路由器IP: 127 . 0 . 0 . 1
- 服务器IP: 127 . 0 . 0 . 1
- 端口: 111
- 服务器端口: 112
- 丢包率: 5 %
- 延时: 5 ms

Buttons: 确定 (OK), 修改 (Modify)

日志 (Log):

```
Delay 5 ms.  
count:9.  
Delay 5 ms.  
count:10.  
Delay 5 ms.  
count:11.  
Delay 5 ms.  
count:12.  
Delay 5 ms.  
count:13.  
Delay 5 ms.
```

这里只以 1.jpg 为例。

Microsoft Visual Studio 调试

2024-12-10 22:19:03 [Send]seq = 128, ack = 129

2024-12-10 22:19:03 127 full load data message received successfully.

2024-12-10 22:19:03 [Send]seq = 130

2024-12-10 22:19:03 [Send]seq = 129, ack = 130

2024-12-10 22:19:03 128 full load data message received successfully.

2024-12-10 22:19:03 [Send]seq = 131

2024-12-10 22:19:03 [Send]seq = 130, ack = 131

2024-12-10 22:19:03 129 full load data message received successfully.

2024-12-10 22:19:03 [Send]seq = 132

2024-12-10 22:19:03 [Send]seq = 131, ack = 132

2024-12-10 22:19:03 130 full load data message received successfully.

2024-12-10 22:19:03 [Send]seq = 133

2024-12-10 22:19:03 [Send]seq = 132, ack = 133

2024-12-10 22:19:03 131 full load data message received successfully.

2024-12-10 22:19:09 [Send]seq = 134

2024-12-10 22:19:09 [Send]seq = 133, ack = 134

2024-12-10 22:19:09 132 full load data message received successfully.

2024-12-10 22:19:09 [Send]seq = 135

2024-12-10 22:19:09 [Send]seq = 134, ack = 135

2024-12-10 22:19:09 Not full load data message received successfully.

2024-12-10 22:19:09 File has written successfully.

2024-12-10 22:19:09 First Handwave succeeded.(Receive)

2024-12-10 22:19:09 Second Handwave succeeded.(Send)

2024-12-10 22:19:09 Third Handwave succeeded.(Send)

2024-12-10 22:19:09 Forth Handwave succeeded.(Receive)

2024-12-10 22:19:09 Server Connection has closed.

D:\Desktop\lab3-Server\Server\x64\Debug\Server.exe (进程 45708)已退出, 代码为 0 (0x0).
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . .

Microsoft Visual Studio 调试

2024-12-10 22:19:03 [Timeout Resend] seq = 134

2024-12-10 22:19:03 [Timeout Resend] seq = 135

2024-12-10 22:19:03 [Receive] ack = 129

2024-12-10 22:19:03 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 2, ssthresh = 1

2024-12-10 22:19:03 [Receive] ack = 130

2024-12-10 22:19:03 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 2, ssthresh = 1

2024-12-10 22:19:03 [Receive] ack = 131

2024-12-10 22:19:03 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 3, ssthresh = 1

2024-12-10 22:19:03 [Receive] ack = 132

2024-12-10 22:19:03 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 3, ssthresh = 1

2024-12-10 22:19:03 [Receive] ack = 133

2024-12-10 22:19:03 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 3, ssthresh = 1

2024-12-10 22:19:08 [Timeout Resend] seq = 134

2024-12-10 22:19:08 [Timeout Resend] seq = 135

2024-12-10 22:19:09 [Receive] ack = 134

2024-12-10 22:19:09 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 2, ssthresh = 1

2024-12-10 22:19:09 [Receive] ack = 135

2024-12-10 22:19:09 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 2, ssthresh = 1

2024-12-10 22:19:09 Total transfer time: 153341ms

2024-12-10 22:19:09 Average throughput: 12.1126 bytes/ms

2024-12-10 22:19:09 First Handwave succeeded.(Send)

2024-12-10 22:19:09 Second Handwave succeeded.(Receive)

2024-12-10 22:19:09 Third Handwave succeeded.(Receive)

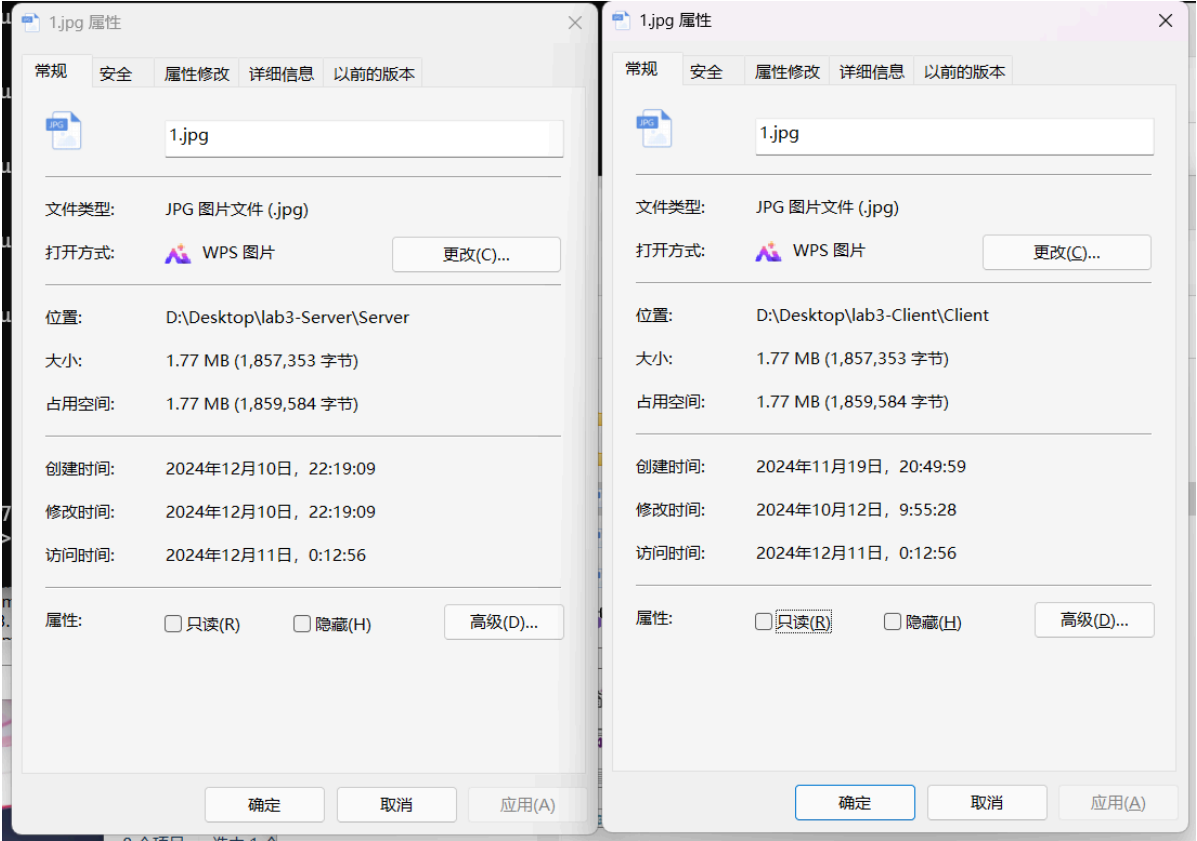
2024-12-10 22:19:09 Forth Handwave succeeded.(Send)

2024-12-10 22:19:09 Client in TIME_WAIT state

2024-12-10 22:19:19 Client successfully closed connection!

D:\Desktop\lab3-Client\Client\x64\Debug\Client.exe (进程 32812)已退出, 代码为 0 (0x0).
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . .

文件确认：



RENO 算法展示:

开始传输时处于慢启动阶段。

```
Microsoft Visual Studio 调试 x + -
2024-12-10 22:16:35 [Window] Window size: 32, has sent but not received confirmation: 28
2024-12-10 22:16:35 [Send] seq = 30, checknum = 27140
2024-12-10 22:16:35 [Window] Window size: 32, has sent but not received confirmation: 29
2024-12-10 22:16:35 [Send] seq = 31, checknum = -3340
2024-12-10 22:16:35 [Window] Window size: 32, has sent but not received confirmation: 30
2024-12-10 22:16:35 [Send] seq = 32, checknum = 17342
2024-12-10 22:16:35 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:35 [Send] seq = 33, checknum = 16829
2024-12-10 22:16:35 [Window] Window size: 32, has sent but not received confirmation: 32
2024-12-10 22:16:36 [Receive] ack = 4
2024-12-10 22:16:36 [Congestion] State: SLOW_START, cwnd = 2, ssthresh = 16
2024-12-10 22:16:36 [Send] seq = 34, checknum = 16257
2024-12-10 22:16:36 [Window] Window size: 32, has sent but not received confirmation: 30
2024-12-10 22:16:36 [Receive] ack = 5
2024-12-10 22:16:36 [Congestion] State: SLOW_START, cwnd = 3, ssthresh = 16
2024-12-10 22:16:36 [Send] seq = 35, checknum = -4486
2024-12-10 22:16:36 [Window] Window size: 32, has sent but not received confirmation: 30
2024-12-10 22:16:36 [Send] seq = 36, checknum = 14214
2024-12-10 22:16:36 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:36 [Receive] ack = 6
2024-12-10 22:16:36 [Congestion] State: SLOW_START, cwnd = 4, ssthresh = 16
2024-12-10 22:16:36 [Send] seq = 37, checknum = 20169
2024-12-10 22:16:36 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:36 [Send] seq = 38, checknum = -260
2024-12-10 22:16:36 [Window] Window size: 32, has sent but not received confirmation: 32
2024-12-10 22:16:36 [Receive] ack = 7
2024-12-10 22:16:36 [Congestion] State: SLOW_START, cwnd = 5, ssthresh = 16
2024-12-10 22:16:36 [Send] seq = 39, checknum = 11914
2024-12-10 22:16:36 [Window] Window size: 32, has sent but not received confirmation: 32
2024-12-10 22:16:41 [Timeout Resend] seq = 8
```

进入拥塞避免阶段。

```
Microsoft Visual Studio 调试 x + -
2024-12-10 22:16:41 [Timeout Resend] seq = 36
2024-12-10 22:16:41 [Timeout Resend] seq = 37
2024-12-10 22:16:41 [Timeout Resend] seq = 38
2024-12-10 22:16:41 [Timeout Resend] seq = 39
2024-12-10 22:16:41 [Receive] ack = 9
2024-12-10 22:16:41 [Congestion] State: SLOW_START, cwnd = 2, ssthresh = 3
2024-12-10 22:16:41 [Send] seq = 40, checknum = -5901
2024-12-10 22:16:41 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:41 [Receive] ack = 10
2024-12-10 22:16:41 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 3, ssthresh = 3
2024-12-10 22:16:41 [Send] seq = 41, checknum = -15211
2024-12-10 22:16:41 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:41 [Receive] ack = 11
2024-12-10 22:16:41 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 3, ssthresh = 3
2024-12-10 22:16:41 [Send] seq = 42, checknum = 11381
2024-12-10 22:16:41 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:41 [Receive] ack = 12
2024-12-10 22:16:41 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 3, ssthresh = 3
2024-12-10 22:16:41 [Send] seq = 43, checknum = -2229
2024-12-10 22:16:41 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:41 [Receive] ack = 13
2024-12-10 22:16:41 [Congestion] State: CONGESTION_AVOIDANCE, cwnd = 4, ssthresh = 3
2024-12-10 22:16:41 [Send] seq = 44, checknum = -21656
2024-12-10 22:16:41 [Window] Window size: 32, has sent but not received confirmation: 31
2024-12-10 22:16:41 [Send] seq = 45, checknum = 28491
2024-12-10 22:16:41 [Window] Window size: 32, has sent but not received confirmation: 32
2024-12-10 22:16:46 [Timeout Resend] seq = 14
2024-12-10 22:16:46 [Timeout Resend] seq = 15
2024-12-10 22:16:46 [Timeout Resend] seq = 16
2024-12-10 22:16:46 [Timeout Resend] seq = 17
```